Modern Programming Principles and Practice
Semester 2
Assessment #3

Module Title: **Modern Programming Principles and Practice**
Assessment Type: Practical Assessment – Individual
Weighting: **40%**
Maximal Possible Mark: 100 marks
Final Submission Date: AS PER MOODLE
Weekly Submission Dates start: AS PER MOODLE

Module Title: **Modern Programming Principles and Practice**
Assessment Type: Practical Assessment – Individual or up to 2 Students collaborating
Weighting: **TBD**
Maximal Possible Mark: 100 marks
Final Submission Date: AS PER MOODLE
Weekly Submission Dates start: AS PER MOODLE
Github Repo – oop-s2-1-mvc-<student-number> e.g. (oop-s2-1-mvc-1234)
Share To: JohnRowleyDorsetCollege

Assignment brief

Acme Global College (VGC) – Multi-Branch Student & Course Management System

Scenario

Acme Global College (VGC) is an established college with **three branches in Ireland**. They currently rely on paper-based processes (Word/Excel) to manage courses, students, and faculty across all branches. You have been assigned to design and implement a **web application** to support **student registration, attendance tracking, and academic progress tracking**, using **mock/seed data**.

Technology constraints

You must implement this as a:

- **ASP.NET Core MVC** application
- Using **Entity Framework Core** with a relational database (SQL Server or SQLite)
- Using **ASP.NET Core Identity** for authentication + role-based authorization
- Using **xUnit** for unit tests
- Using **GitHub** for source control and **GitHub Actions** for CI (build + test)

Uploading exam scripts is **not required**. You are tracking exam definitions and results only.

Core requirements

Users and access rules (RBAC)

Your system must support three roles:

1. **Administrator**
    - Can create and manage: branches, courses, modules (optional), timetables (simplified), users, and enrolments
    - Can view all data
2. **Faculty**
    - Can only view/manage information relevant to **their assigned courses / students**
    - Must be able to access:
        - A student **gradebook** for their students (assignment results)
        - **Student contact details** for their students (tutors only)
3. **Student**
    - Can login and view **only their own** information
    - Must NOT be able to see **provisional exam results** until released (simple boolean flag is fine)

Functional scope

A) Student registration & enrolment

- Create/view/edit student profile (contact + identifying info)
- Enrol a student in a course (or course offering)
- Track ongoing attendance for enrolled students (simplified weekly sessions is fine)

B) Academic progress tracking

- Store assignment results (gradebook)
- Store exam results (no upload)
- Support “released” vs “provisional” exam result visibility

C) Faculty views

- Faculty can see:
  - Their students by course
  - Gradebook entries they manage
  - Contact details for students they tutor/teach (as per your rules)

D) Admin management

- Manage courses (and optionally modules)
- Manage staff assignments (faculty to course)
- Manage enrolments
- Manage result release (e.g., ReleaseExamResults = true)

Data model (minimum suggested entities)

You may adjust, but your design must support the requirements.

- **Branch** (Id, Name, Address)
- **Course** (Id, Name, BranchId, StartDate, EndDate)
- **StudentProfile** (Id, IdentityUserId, Name, Email, Phone, Address, DOB or StudentNumber)
- **FacultyProfile** (Id, IdentityUserId, Name, Email, Phone)
- **CourseEnrolment** (Id, StudentProfileId, CourseId, EnrolDate, Status)
- **AttendanceRecord** (Id, CourseEnrolmentId, WeekNumber/Date, Present)
- **Assignment** (Id, CourseId, Title, MaxScore, DueDate)
- **AssignmentResult** (Id, AssignmentId, StudentProfileId, Score, Feedback)
- **Exam** (Id, CourseId, Title, Date, MaxScore, ResultsReleased bool)
- **ExamResult** (Id, ExamId, StudentProfileId, Score, Grade)

Mandatory non-functional requirements

- **Authorization is enforced server-side** (not just hidden UI links)
- **Validation** (data annotations + server-side checks)
- **Error handling** (friendly error pages, no raw exception leaks)
- **Seed/mock data** so the app is usable immediately after database creation
- **README** that explains setup, login accounts, and how to run tests

---

Deliverables

Your GitHub repository must contain:

1. /src/VgcCollege.Web (MVC app)
2. /tests/VgcCollege.Tests (xUnit test project)
3. .github/workflows/ci.yml (build + test workflow)
4. README.md with:
    - How to run the app locally
    - How to run tests
    - Seeded demo accounts (Admin / Faculty / Student)
    - Any design decisions / assumptions

Marking scheme (with CI + GitHub deployment)
Overview (100%)

Category	Marks
1. Core features implemented	35
2. Security & authorization (RBAC)	20
3. Data & persistence (EF Core + migrations + seed)	15
4. Testing (xUnit)	15
5. GitHub: repo quality + CI workflow	10
6. UX, validation, error handling, polish	5
Total	**100**

---

1) Core features implemented (35)
Student registration & enrolment (15)

- (5) Create/Edit student profile + validation
- (5) Enrol student into course; list enrolments by student/course
- (5) Attendance tracking per enrolment/session (create + view)

Academic progress (20)

- (8) Assignment gradebook: assignments + results entry/viewing
- (8) Exams + exam results tracking (no uploads) + editing
- (4) "Results released" rule works (students cannot view provisional results)

---

2) Security & authorization (20)

- (4) Identity authentication implemented correctly (login/register not required for all roles if seeded accounts used, but login must work)
- (10) Role-based authorization enforced on controllers/actions (Admin/Faculty/Student)
- (6) Data access restricted correctly (Faculty only sees their students; Student only sees self)

Full marks require server-side enforcement (policy/roles/queries filtered by user), not only UI hiding.

---

3) Data & persistence (15)

- (6) EF Core entities + relationships correct and normalized
- (5) Migrations included and database builds cleanly
- (4) Seed/mock data meaningful (3 branches, multiple courses, faculty assignments, enrolments, results)

---

4) Testing (15)

- (5) At least **8 unit tests** for domain/service logic (not just trivial "true is true" tests)
- (6) Tests cover: enrolment rules, visibility rules, grade calculations/validation, or authorization-related query filtering (as appropriate)
- (4) Tests are deterministic and run in CI (e.g., using SQLite in-memory or EFCore InMemory provider where appropriate)

---

5) GitHub repo quality + CI workflow (10)

- (3) Repository structure is clean (src/, tests/, sensible naming)
- (3) README complete and accurate (setup + seeded credentials + run instructions)
- (4) GitHub Actions workflow:
    - Restores
    - Builds in **Release**
    - Runs tests
    - Fails the check if build/tests fail

---

6) UX, validation, error handling, polish (5)

- (2) Clear navigation per role (Admin/Faculty/Student)
- (2) Validation messages + no broken flows
- (1) Reasonable styling/layout and consistent pages

GitHub Actions CI workflow (required)
Create this file:
/.github/workflows/ci.yml

```yaml
name: ci

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "8.0.x"

      - name: Restore
        run: dotnet restore

      - name: Build (Release)
        run: dotnet build --configuration Release --no-restore

      - name: Test
        run: dotnet test --configuration Release --no-build --verbosity normal
```

Marking rule for CI:
If the workflow is missing or failing on main, cap the “GitHub repo quality + CI workflow” category at **0/10**.

Practical guidance for student

Minimum seeded accounts

Seed at least:

- 1 Admin user
- 1 Faculty user
- 2 Student users

  …and make credentials available in README.md.

Suggested milestones

- **Milestone 1:** Auth + roles + database created + seed data
- **Milestone 2:** Enrolments + attendance
- **Milestone 3:** Gradebook + exams + release rules
- **Milestone 4:** Tests + CI + final polish